Auriseg Penentration Assessment Report



This report documents a completed black-box web application penetration test of the Chola Insurance UAT Portal. The primary risk is unauthenticated access to sensitive static resources outside the application's dynamic authentication boundary.

Assessment Information

Field	Detail
Target	`https://uatportal.cholainsurance.com/epolcommon_ggpoc/`
Assessment type	Black-box web application penetration test
Environment	UAT
Assessment date	September 2026
Testing window	10:00–18:00 IST
Assessment status	Completed

1. Executive Summary

A black-box security assessment was performed against the Chola Insurance UAT portal.

The assessment identified multiple **unauthenticated exposure issues**, primarily involving business documents, member information, internal application components, directory listings, and verbose application errors.

The most significant issue is that sensitive business files were directly accessible without authentication despite the application having an authentication mechanism for its dynamic pages.

Confirmed Findings

ID	Finding	Severity
F-01	Unauthenticated exposure of member/business data	**High**
F-02	Unauthenticated exposure of bulk business files	**High**
F-03	Public directory listing / resource enumeration	**Medium**
F-04	Public exposure of server-side application DLLs	**Medium**
F-05	Verbose ASP.NET error messages exposed	**Medium**
F-06	Internal build-path metadata disclosure	**Low**

The assessment also identified several security controls that were functioning as expected, including HTTPS/TLS, HSTS, ASP.NET ViewState integrity protection, authentication redirects on several application pages, and rejection of several attempted attack techniques.

2. Critical Observation

The primary security problem is **not simply that files exist on the server**.

The problem is that the files are available through the web server **without authentication or authorization checks**.

For example:

```
GET /epolcommon_ggpoc/docs/UWrenewal.xlsx HTTP/1.1
Host: uatportal.cholainsurance.com
```

returned:

```
HTTP/1.1 200 OK
Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
Content-Length: 11515
```

No authenticated session was required. The same pattern was observed for other business files and server-side application assemblies.

3. Finding F-01 — Unauthenticated Exposure of Member / Business Data

Severity: High
OWASP: A01 – Broken Access Control
CWE: CWE-862 / CWE-200
Affected resource: `/epolcommon_ggpoc/docs/FacSlipTemplate/Sample_Member_Details_File_Bulk.xls`

Description

A business/member workbook was directly accessible without authentication. The file was retrieved using a normal HTTP GET request without supplying an authenticated session.

Proof of Concept

```
curl -sk -o /tmp/Sample_Member_Details_File_Bulk.xls \\
  "https://uatportal.cholainsurance.com/epolcommon_ggpoc/docs/FacSlipTemplate/Sample_Member_Details_File_Bulk.xls"

file /tmp/Sample_Member_Details_File_Bulk.xls
```

Observed response:

```
HTTP 200
Content-Type: application/vnd.ms-excel
Content-Length: 60416
```

The workbook was successfully parsed. It contained:

```
Sheet: Sheet1
Rows: 301
Columns: 6
```

The columns included:

```
SRNO
NAME_OF_EMPLOYEE
GENDER
SUM_INSURED
DATE_OF_BIRTH
RELATIONSHIP
```

The workbook contained populated records rather than being an empty template. The assessment did not reproduce the complete records in this report.

Additional Evidence

The same file was accessible directly without a cookie:

```
curl -sk -b '' -I \\
  "https://uatportal.cholainsurance.com/epolcommon_ggpoc/docs/FacSlipTemplate/Sample_Member_Details_File_Bulk.xls"
```

The file was returned successfully.

SHA-256:

```
0c7439e09c470998bbd378aa3ad589f7a16a6d824eb55966e97192c2c1632
```

Impact

An unauthenticated attacker who discovers or guesses the URL can obtain the workbook directly.

Depending on whether the exposed file represents production-like, operational, or sample business data, consequences may include:

- Disclosure of employee/member information.
- Exposure of personal information.
- Exposure of insurance-related information.
- Privacy violations.
- Unauthorized data harvesting.
- Phishing or social-engineering opportunities.
- Regulatory/compliance exposure.
- Reputational damage.

The important issue is that the attacker does **not** need to successfully authenticate to obtain the file.

Root Cause

The file appears to be stored beneath a web-accessible directory and is being served directly by IIS.

The application authentication mechanism protects dynamic pages such as:

```
/report.aspx
/search.aspx
/home.aspx
```

but does not prevent direct access to the static workbook.

Recommendation — Immediate

Remove sensitive business/member files from the web root.

Prefer:

```
Application
   |
   +-- Authenticated request
           |
           v
       Authorization
           |
           v
     File retrieval
```

rather than:

```
Internet
   |
   v
/static-file.xls
   |
   v
HTTP 200
```

Recommendation — Long Term

- Store sensitive files outside the IIS web root.
- Require authentication before file retrieval.
- Perform authorization checks against the requesting user's permissions.
- Remove obsolete sample/member files.
- Review the entire `/docs/` tree for similar exposure.
- Review IIS virtual directories and static-file mappings.

- Prevent direct public access to `.xls`, `.xlsx`, `.csv`, `.pdf`, `.doc`, `.docx`, `.txt`, and similar business artifacts where appropriate.

4. Finding F-02 — Unauthenticated Exposure of Bulk Business Files

Severity: High
OWASP: A01 – Broken Access Control
CWE: CWE-862 / CWE-200
Affected resource: `/epolcommon_ggpoc/bulksms/UploadFolder/`

Description

The application exposes an IIS directory containing bulk business files. The directory itself is publicly accessible.

```
GET /epolcommon_ggpoc/bulksms/UploadFolder/
```

returned:

```
HTTP 200
```

The directory contained **151 uniquely named `.xls` files**. The listing response was approximately **19,775 bytes**.

Proof of Concept

```
curl -sk \\
  "https://uatportal.cholainsurance.com/epolcommon_ggpoc/bulksms/UploadFolder/"
```

The response contained links to numerous `.xls` files.

A representative file was then retrieved without authentication:

```
/epolcommon_ggpoc/bulksms/UploadFolder/010025180035925.xls
```

Observed:

```
HTTP 200
Content-Type: application/vnd.ms-excel
Content-Length: 27136
```

SHA-256:

```
94131ad69b4b871fae2190f367e7b44b5290e3f84e31715b243b0ecc43a4e822
```

The workbook contained a `BulkSMS` sheet with a `POLICY_NO` field.

Impact

The combination of:

1. Public directory listing.
2. Predictable/directly addressable files.
3. No authentication requirement.

creates a scalable information-disclosure condition.

An attacker does not need to compromise the application first.

Potential consequences include:

- Unauthorized access to insurance-related files.
- Disclosure of policy-related information.
- Bulk harvesting of business records.
- Privacy exposure.

- Information useful for targeted phishing.
- Increased attack-surface knowledge.
- Exposure of historical operational data.

The assessment intentionally did **not** download all 151 files. One representative file was sufficient to establish the exposure.

Recommendation

Immediately:

```
Disable directory browsing
        +
Remove unnecessary files
        +
Move sensitive files outside web root
        +
Require authorization for downloads
```

IIS directory browsing should not be enabled for directories containing business data.

The upload/download architecture should be reviewed so that uploaded documents are not automatically exposed as static web resources.

Where files must remain on the server:

- Store them outside the publicly accessible web directory.
- Assign opaque identifiers.
- Use an authenticated download handler.
- Perform authorization before returning the file.
- Log file access.
- Delete obsolete files according to retention requirements.

5. Finding F-03 — Public Directory Listing and Resource Enumeration

Severity: Medium
OWASP: A05 – Security Misconfiguration
CWE: CWE-548

Description

Multiple directories expose IIS directory listings to unauthenticated users.

Confirmed examples include:

```
/docs/
/libraries/
/js/
/css/
/images/
/packages/
/bulksms/
/bulksms/UploadFolder/
/uploadfiles/
/docs/FacSlips/
```

Example

Request:

```
GET /epolcommon_ggpoc/docs/ HTTP/1.1
```

returned:

```
HTTP 200
```

with an IIS directory listing.

The `/docs/` listing exposed:

```
ErrorLog.txt
FacSlips/
FacSlipTemplate/
UWrenewal.xlsx
```

Similarly:

```
GET /epolcommon_ggpoc/libraries/ HTTP/1.1
```

returned a directory listing containing server-side assemblies.

Impact

Directory browsing significantly reduces the effort required to map an application.

An attacker can discover:

- File names.
- Directory structure.
- Application components.
- Deployment artifacts.
- Business documents.
- Log files.
- Framework libraries.
- Historical files.
- Potentially sensitive operational resources.

In this assessment, directory listing directly assisted discovery of the higher-impact findings.

Recommendation

Disable directory browsing at IIS/application level.

For example, the desired behavior for sensitive directories should be:

```
GET /docs/
        |
        v
403 Forbidden
```

rather than:

```
GET /docs/
        |
        v
200 OK
Directory listing
```

Additionally:

- Remove unnecessary files.
- Remove obsolete deployment artifacts.
- Restrict static directories.
- Review IIS configuration.
- Separate public static content from private application data.

6. Finding F-04 — Public Exposure of Server-Side Application Assemblies

Severity: Medium
OWASP: A05 – Security Misconfiguration
CWE: CWE-200

Description

The `/libraries/` directory exposes compiled `.NET` assemblies directly to unauthenticated users.

Observed application-specific assemblies include:

```
BancaCommon.dll
CHDBLayer.dll
CHUtils.dll
```

along with third-party libraries.

Example:

```
/libraries/CHUtils.dll
```

was directly downloadable.

Proof of Concept

```
curl -sk -o /tmp/CHUtils.dll \\
  "https://uatportal.cholainsurance.com/epolcommon_ggpoc/libraries/CHUtils.dll"
```

Observed:

```
HTTP 200
Content-Type: application/x-msdownload
Content-Length: 19456
```

The downloaded file was identified as a PE32 .NET assembly.

Another application-specific assembly, `CHDBLayer.dll`, was also directly accessible. Observed size: **29,184 bytes**.

Static Analysis

Analysis of `CHDBLayer.dll` revealed application-specific functionality including:

```
CHLDAP
AuthenticateUser
IsLDAPUserExists
```

The assembly also contained references to:

```
System.Data
System.Data.OracleClient
System.Data.OleDb
System.DirectoryServices
System.DirectoryServices.AccountManagement
```

The LDAP implementation included methods involving:

```
PrincipalContext
DirectoryEntry
DirectorySearcher
UserPrincipal
SAMAccountName
```

This demonstrates that the exposed DLL contains meaningful application implementation details.

It did **not**, however, demonstrate that LDAP authentication itself was remotely exploitable. No concrete credentials, LDAP server address, or database credentials were identified in the tested assemblies.

Impact

Exposing application binaries provides an attacker with implementation information that would normally be unavailable to an external user.

Possible consequences include:

- Reverse engineering of application functionality.
- Discovery of internal class/method names.
- Discovery of authentication implementation details.
- Identification of database technologies.
- Identification of directory-service integration.
- Improved reconnaissance for future attacks.
- Easier identification of vulnerable code paths.
- Exposure of internal architectural information.

The DLL exposure should therefore be considered an information-disclosure and attack-surface reduction issue.

Recommendation

Compiled application assemblies should not be directly downloadable from the public web root.

Recommended architecture:

```
Internet
   |
   v
IIS
   |
   +-- Public static resources only
   |
   +-- Application runtime
          |
          +-- Private assemblies
```

rather than:

```
Internet
   |
   v
/libraries/*.dll
```

Recommended actions:

- Move assemblies outside the public web directory where practical.
- Deny direct HTTP access to `.dll`, `.pdb`, `.xml`, `.config`, and other deployment artifacts.
- Remove unused libraries.
- Apply appropriate IIS request filtering.
- Deploy only required runtime files.
- Ensure debug/build artifacts are excluded from production/UAT web exposure where not required.

7. Finding F-05 — Verbose ASP.NET Error Messages Exposed

Severity: Medium
OWASP: A05 – Security Misconfiguration
CWE: CWE-209

Description

A malformed login request caused the application to return a detailed ASP.NET exception page to the client.

A controlled request using a modified username produced an HTTP 500 response with a verbose ASP.NET error response.

The response disclosed an exception relating to:

```
Validation of viewstate MAC failed.
```

The response additionally exposed internal ASP.NET configuration guidance indicating that verbose error messages were enabled. It referenced:

```
customErrors mode="Off"
```

as the relevant configuration condition.

Example Evidence

The response contained an exception of the form:

```
System.Web.HttpException:
Validation of viewstate MAC failed.
```

It also disclosed implementation/environment details including the possibility of a web-farm configuration involving:

```
<machineKey>
validationKey
validation algorithm
```

Impact

Verbose errors provide attackers with information that can assist further reconnaissance.

Potentially exposed information includes:

- Framework implementation.
- Application configuration.
- Exception types.
- Internal processing behavior.
- Security-control configuration.
- Deployment architecture.
- Stack-trace information.
- Implementation details useful for targeted attacks.

The issue becomes more significant when combined with the other information disclosures already present in the application.

Recommendation

Production-style deployments should not return detailed ASP.NET exception information to remote clients.

Recommended configuration:

```
<system.web>
    <customErrors mode="On" />
</system.web>
```

or an appropriately controlled:

```
<system.web>
    <customErrors mode="RemoteOnly" />
</system.web>
```

depending on operational requirements.

Detailed exceptions should instead be returned through the following pattern:

```
Application
    |
    +--> Generic client error
    |
    +--> Detailed server-side log
```

For example, an external client should receive:

```
HTTP 500
An unexpected error occurred.
```

while the full diagnostic information is retained securely in server-side logging.

The exposed `/docs/ErrorLog.txt` should also be removed from public access.

8. Finding F-06 — Internal Build-Path / PDB Metadata Disclosure

Severity: Low
OWASP: A05 – Security Misconfiguration
CWE: CWE-200

Description

The publicly downloadable .NET assemblies contain internal build/debug metadata revealing developer workstation and source/build paths.

Examples identified included:

```
E:\bitbucket\chpower\V4.6\CHUtils\CHUtils\obj\Debug\CHUtils.pdb
E:\bitbucket\chpower\V4.6\CHDBLayer\CHDBLayer.pdb
```

Other exposed metadata referenced paths associated with:

```
F:\TESTING\Asp.Net3.5\AjaxControlToolkit\
D:\Haribabu\CHPower\trunk\V4.0\
c:\MyFiles\Projects\ClosedXML\
d:\oxapi\oxapiV0\
D:\github\npoi\
```

Important Qualification

The PDB files themselves were **not** confirmed to be publicly accessible.

For example:

```
/libraries/CHDBLayer.pdb
```

returned:

```
HTTP 404
```

Therefore, this finding concerns metadata contained in exposed binaries, not direct PDB-file exposure.

Impact

Internal build paths can reveal:

- Developer workstation naming.
- Source repository structure.
- Project names.
- Software versions.
- Build configurations.
- Directory conventions.

- Development environment details.

This information has limited direct impact but can improve attacker reconnaissance.

Recommendation

Build production artifacts without unnecessary debugging metadata.

Recommended controls include:

- Build Release artifacts.
- Remove PDB/debug information from public deployment directories.
- Prevent public access to `.pdb` files.
- Review compiler/debug settings.
- Avoid publishing development artifacts.
- Keep source/build information within internal build systems.

9. Supporting Finding — `/docs/ErrorLog.txt`

The public `/docs/` directory exposed:

```
ErrorLog.txt
```

Observed:

```
HTTP 200
Content-Type: text/plain
Size: 3147 bytes
```

SHA-256:

```
8b139e44c98f1c75bb3f8e68f14a0fe132cc5c68e0708e2e94fc1febd4236adf
```

The file contained detailed ASP.NET/.NET exception information including classes and methods such as:

```
System.AggregateException
System.Net.Http.HttpRequestException
System.Net.WebException
System.IO.IOException
System.Net.Sockets.SocketException

DSEnachPayCtrlr.SaveEnatchOTP_Netcore(...)
OpsSendSMS.btnSendSMS_Click(...)
```

This evidence supports the information-disclosure and verbose-error findings. The log should not be publicly accessible.

10. Additional Exposed Resources

The assessment identified other publicly browsable directories.

`/js/`

The directory exposed JavaScript libraries including:

```
jquery
bootstrap
jquery-ui
BulkCertificateCtrlr.js
```

No additional exploitable endpoint was demonstrated from the reviewed client-side JavaScript.

`/css/`

The directory exposed CSS resources. No direct vulnerability was demonstrated solely from this exposure.

`/images/`

The directory exposed image files, including administrative-themed image assets. No credential or authentication functionality was established from these images.

`/packages/`

The directory exposed package-related resources including:

```
NPOI.Excel.2.1.1/
```

A signature artifact was directly retrievable:

```
.signature.p7s
```

No standalone security vulnerability was demonstrated from the package signature artifact.

11. Third-Party Component Observations

Several third-party components were identified in the publicly accessible `/libraries/` directory.

Examples include:

```
Newtonsoft.Json.dll
AjaxControlToolkit.dll
NPOI.dll
NPOI.OOXML.dll
ClosedXML.dll
DocumentFormat.OpenXml.dll
ICSharpCode.SharpZipLib.dll
Ionic.Zip.dll
itextsharp.dll
```

Observed versions included:

```
Newtonsoft.Json
Assembly Version: 9.0.0.0
Observed version string: 9.0.1.19813

AjaxControlToolkit
Version: 3.0.20229.32551
```

These versions should be reviewed against the organization's approved dependency baseline and current vendor security advisories.

However, no specific third-party CVE was treated as a confirmed exploitable vulnerability during this assessment.

For example, the known AjaxControlToolkit file-upload handler path was checked and:

```
/AjaxFileUploadHandler.axd
```

returned:

```
HTTP 404
```

Therefore, component-version observations should be treated as remediation items requiring dependency review rather than confirmed remote exploits.

12. Authentication Testing Results

The application implements an authentication mechanism.

The login page contains:

```
txtUserID
txtPassword
BtnLogin
```

Unauthenticated access to several dynamic pages was correctly redirected.

Examples:

```
/report.aspx
/search.aspx
/home.aspx
```

redirected to:

```
/Login.aspx
```

This indicates that authentication controls exist for those application pages.

However, static resources underneath publicly accessible directories were not consistently protected. This difference is the primary access-control issue.

13. Session Cookie Observation

During a normal Web Forms login interaction, the application issued:

```
ASP.NET_SessionId
```

with:

```
Secure
HttpOnly
SameSite=Lax
```

A second cookie was observed:

```
TS019f4eeb
```

which did not include the same security attributes.

The role and security significance of this cookie were not conclusively established during the assessment. Therefore it is **not classified as a standalone vulnerability** in this report.

14. Security Controls That Were Validated

Several tests did not demonstrate exploitable vulnerabilities.

ViewState Integrity

Tampering with ViewState resulted in:

```
The state information is invalid for this page and might be corrupted.
```

This indicates that ViewState integrity validation was functioning.

SQL Injection / Authentication Bypass

A controlled authentication-bypass payload:

```
' OR '1'='1'--
```

did not bypass authentication.

The application returned the normal invalid-login response. No SQL error was observed. SQL injection was therefore **not demonstrated**.

Reflected Input

Input such as:

```
test'
```

was HTML encoded in the response:

```
test&#39;
```

No reflected XSS was demonstrated.

Path Traversal

Controlled traversal requests such as:

```
../../../../Windows/win.ini
```

and encoded variants did not return file contents.

The application returned HTTP 404 or normal application content. Path traversal was therefore **not demonstrated**.

Popup File Parameter

The endpoint:

```
/popup.aspx?file=...
```

was tested with controlled values.

The parameter was reflected in the form action, but no server-side file contents were obtained. The image element remained:

```
<img id="image" src="" border="0" />
```

No arbitrary file read was demonstrated.

CORS

The application returned an unusual value:

```
Access-Control-Allow-Origin: https://uatportal.cholainsurance.com/epolcommon_ggpoc/
```

A request using an unrelated origin did not cause the server to reflect that attacker-controlled origin. A practical CORS exploitation scenario was therefore not demonstrated.

15. TLS / Transport Security

The target supported:

```
TLS 1.2
TLS 1.3
```

TLS 1.0 and TLS 1.1 were not observed as supported.

HSTS was present:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

The target also returned:

```
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
```

No major transport-security vulnerability was demonstrated.

16. Overall Attack Scenario

The most realistic attack path identified during testing does not require an authentication bypass. It is substantially simpler:

```
Internet
   |
   v
Target URL
   |
   v
Directory enumeration
   |
   +----------------------+
   |                      |
   v                      v
/docs/              /bulksms/UploadFolder/
   |                      |
   v                      v
Business files       151 XLS files
   |
   +----------------------+
   |
   v
/libraries/
   |
   v
.NET assemblies
   |
   v
Internal implementation information
```

The attacker can therefore obtain meaningful information without first compromising the login mechanism.

17. Business Impact Summary

The combined impact is greater than any single directory listing. The exposed resources can provide three different classes of information.

Business Data

Examples:

```
Policy-related information
Member/employee information
Insurance-related workbook data
```

Potential impact:

```
Privacy
Confidentiality
Regulatory exposure
Targeted phishing
Reputational damage
```

Application Intelligence

Examples:

```
DLLs
Class names
Method names
LDAP implementation
```

```
Database technology references
Build paths
```

Potential impact:

```
Improved attacker reconnaissance
Targeted vulnerability research
Reduced effort for future attacks
```

Operational Information

Examples:

```
Error logs
Directory structure
File names
Historical artifacts
```

Potential impact:

```
Technology fingerprinting
Internal architecture disclosure
Identification of sensitive resources
```

18. Root-Cause Analysis

The findings appear to share several configuration/deployment problems.

Root Cause 1 — Sensitive Data Stored Under Web Root

Business files appear to be deployed beneath publicly accessible paths.

Root Cause 2 — Directory Browsing Enabled

IIS directory browsing makes the underlying resources discoverable.

Root Cause 3 — Static Resources Not Covered by Authorization

The application protects certain dynamic pages but static files can bypass the application authentication boundary.

Root Cause 4 — Development/Operational Artifacts Deployed to Web Server

Examples include:

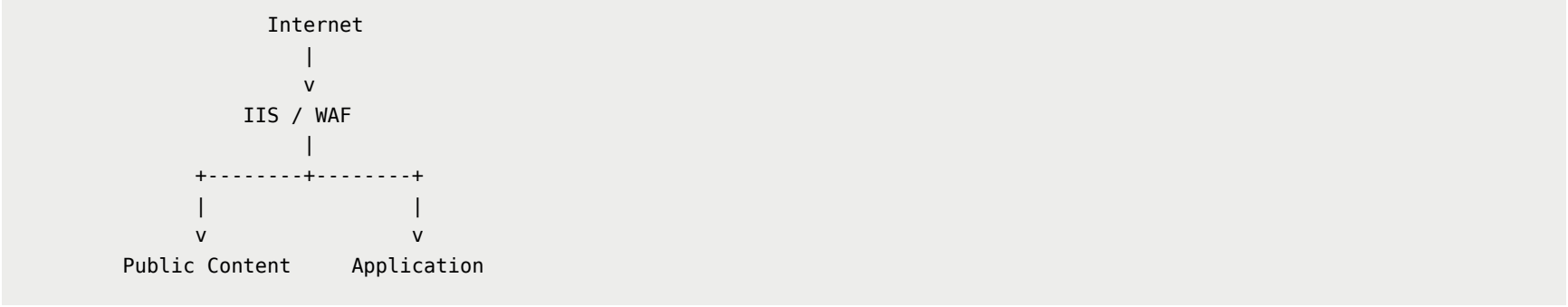
```
ErrorLog.txt
DLLs
XML documentation
business workbooks
historical files
```

Root Cause 5 — Verbose Error Configuration

Detailed ASP.NET errors are exposed to the requesting client.

19. Recommended Remediation Architecture

A corrected architecture should resemble:



```
                          |
                          v
                   Authentication
                          |
                          v
                    Authorization
                          |
                          v
                Secure File Handler
                          |
                          v
              Private File Storage
```

Sensitive files should **not** follow:

```
Internet
   |
   v
IIS
   |
   v
/docs/file.xlsx
```

Instead:

```
User
 |
 v
Authenticated application
 |
 v
Authorization check
 |
 +---- allowed ----> private storage
 |
 +---- denied ------> HTTP 403
```

20. Remediation Priority

Immediate — Highest Priority

1. Remove Public Business Files

Review and remove the following from public web access where business data is present:

```
/docs/
/docs/FacSlipTemplate/
/bulksms/UploadFolder/
```

2. Disable Directory Browsing

Disable IIS directory browsing for the application.

3. Protect File Downloads

Require authentication and authorization for business documents.

4. Remove Public Error Logs

Immediately remove the following from public access:

```
/docs/ErrorLog.txt
```

21. Short-Term Remediation

Application

- Review all static file paths.
- Implement authorization around file retrieval.
- Move sensitive files outside the web root.
- Remove unused historical artifacts.
- Review upload/download workflows.
- Ensure uploaded files cannot be directly retrieved anonymously.

IIS

- Disable directory browsing.
- Add request filtering for sensitive extensions.
- Review virtual directories.
- Restrict direct access to deployment artifacts.
- Prevent public access to debug files.

Error Handling

Configure:

```
<system.web>
    <customErrors mode="On" />
</system.web>
```

and ensure detailed errors are logged server-side rather than returned to clients.

22. Medium-Term Remediation

Perform a complete server-side deployment review.

Search the deployed web root for:

```
*.xls
*.xlsx
*.csv
*.pdf
*.doc
*.docx
*.txt
*.log
*.dll
*.pdb
*.config
*.bak
*.old
*.zip
*.nupkg
```

Each artifact should be classified as:

```
Required public resource
Required private resource
Temporary resource
Development artifact
Obsolete resource
```

Anything not explicitly required for public access should be removed or protected.

23. Dependency Remediation

Review all exposed third-party assemblies against the organization's approved software inventory.

Particular attention should be given to:

```
Newtonsoft.Json
AjaxControlToolkit
```

```
NPOI
ClosedXML
SharpZipLib
iTextSharp
```

The objective should be:

```
Inventory
   |
   v
Identify exact version
   |
   v
Compare with supported version
   |
   v
Review vendor advisories
   |
   v
Upgrade where required
   |
   v
Regression testing
```

Component exposure alone should not be treated as proof of exploitation.

24. Retest Requirements

After remediation, the following tests should be repeated.

Anonymous File Access

Attempt:

```
GET /docs/UWrenewal.xlsx
GET /docs/FacSlipTemplate/Sample_Member_Details_File_Bulk.xls
GET /bulksms/UploadFolder/
GET /libraries/CHUtils.dll
GET /docs/ErrorLog.txt
```

Expected result:

```
403 Forbidden
```

or:

```
404 Not Found
```

depending on the organization's chosen concealment strategy.

Directory Browsing

Attempt:

```
GET /docs/
GET /libraries/
GET /bulksms/
GET /bulksms/UploadFolder/
GET /js/
GET /css/
```

Expected result:

```
No directory listing
```

Error Handling

Repeat malformed requests.

Expected result:

```
HTTP 4xx/5xx
Generic error
No stack trace
No internal path
No configuration disclosure
```

DLL Exposure

Attempt:

```
GET /libraries/CHUtils.dll
GET /libraries/CHDBLayer.dll
```

Expected result:

```
403 / 404
```

unless the resource is explicitly intended to be public.

25. Final Finding Summary

ID	Finding	Severity	Primary risk
F-01	Member/business data directly accessible without authentication	**High**	Confidentiality / privacy
F-02	151 bulk business files exposed through public directory	**High**	Bulk information disclosure
F-03	IIS directory browsing enabled	**Medium**	Reconnaissance / resource discovery
F-04	Server-side .NET DLLs publicly downloadable	**Medium**	Application intelligence disclosure
F-05	Verbose ASP.NET errors exposed	**Medium**	Internal implementation disclosure
F-06	Internal build paths disclosed through DLL metadata	**Low**	Reconnaissance

26. Overall Conclusion

The assessment identified a recurring **public-resource exposure problem** in the application deployment.

The application's dynamic authentication boundary is functioning for several discovered pages, but sensitive static resources are outside that boundary.

The highest-priority remediation is therefore not an authentication algorithm change. It is to **correct the web-server/application deployment model so sensitive files cannot be retrieved anonymously**.

The immediate remediation sequence should be:

```
1. Remove public business/member files
                ↓
2. Disable IIS directory browsing
                ↓
3. Move sensitive files outside web root
                ↓
4. Enforce authorization on file downloads
                ↓
5. Remove public logs and deployment artifacts
                ↓
6. Disable verbose ASP.NET errors
                ↓
7. Restrict direct DLL/debug-artifact access
                ↓
8. Review and update exposed dependencies
                ↓
9. Perform full retest
```

The most significant evidence obtained during the assessment was the ability to retrieve populated business/member workbooks and numerous bulk business files **without authentication**. This should be treated as the primary remediation priority.